МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №4
по курсу «Операционные системы»

Взаимодействие процессов через разделяемую память POSIX

Выполнил: С. М. Закатин
Группа: М8О-203Б-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

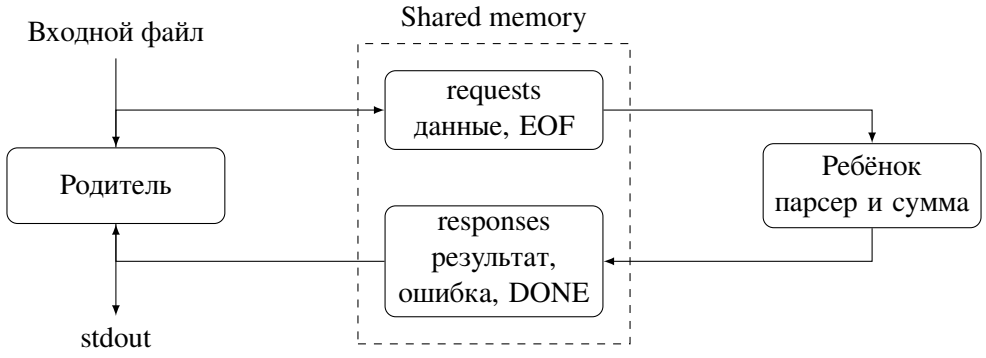
Цель работы: приобретение практических навыков создания процессов и организации двустороннего межпроцессного обмена через разделяемую память POSIX.

Задание: реализовать две отдельные программы на языке Си. Родительский процесс открывает входной файл, создаёт разделяемую область памяти и запускает дочернюю программу с помощью `fork` и `execv`. Содержимое файла передаётся дочернему процессу через очередь в общей памяти. Ребёнок вычисляет сумму целых чисел каждой непустой строки и возвращает результат через вторую очередь. Родитель выводит результаты в стандартный поток вывода. Требуется обрабатывать системные ошибки, переполнение и завершение другого процесса. Поддерживается POSIX; реализация для Windows не требуется.

Вариант: 6, задача суммирования целых чисел из предыдущей работы. Числа разделяются пробельными символами, их количество произвольно.

Метод решения

Программа сохраняет схему создания процессов предыдущей лабораторной: `fork` создаёт ребёнка, после чего `execv` заменяет его образ программой `child`. Изменён транспорт прикладных данных. Вместо передачи результатов через канал используются две независимые SPSC-очереди в одном объекте `shm_open`, отображённом с флагом `MAP_SHARED`. SPSC означает одного производителя и одного потребителя для каждой очереди. В очереди запросов производителем является родитель, в очереди результатов - ребёнок.



Родитель сначала читает готовые результаты, затем пытается отправить очередной блок. Такое чередование предотвращает взаимную блокировку: из одного блока ребёнок может получить больше строк, чем помещается в очереди результатов. Если родитель ожидал бы только освобождения очереди запросов, ребёнок мог бы одновременно ожидать места для ответов.

Для входных данных применяется интерфейс `acquire/commit`. Родитель получает указатель на свободный слот, вызывает `read` прямо для массива байт этого слота и затем публикует индекс записи. Поэтому нет промежуточного копирования из локальной структуры родителя в очередь. До `commit` потребитель не видит заполняемый слот; при ошибке чтения слот не публикуется. Ребёнок получает указатель на опубликованный блок через `spsc_peek_slot`, разбирает байты непосредственно в общей памяти и освобождает слот вызовом `spsc_consume_slot` после обработки всего блока. До освобождения производитель не может занять этот слот повторно. Таким образом, промежуточных копий входного

блока в пользовательской памяти нет ни у родителя, ни у ребёнка; системный `read` по-прежнему копирует данные из ядра. В измеренной версии размер блока равен 4096 байт.

Кольцевая очередь содержит 256 слотов, из которых доступны 255. Один слот оставлен свободным для различения полного и пустого состояний:

$$\text{empty} : r = w, \qquad \text{full} : (w + 1) \bmod 256 = r.$$

Индекс записи изменяет только производитель, индекс чтения - только потребитель. CAS и мьютекс для изменения индексов не требуются. После заполнения слота производитель публикует новый индекс операцией `release`; потребитель наблюдает его через `acquire`. После обработки или копирования сообщения потребитель release-записью освобождает слот, который производитель проверяет через acquire. Собственный индекс читается с `relaxed`. Таким образом, обычный массив сообщений защищён порядком публикации и повторного использования слотов. Требование всегда lock-free атомиков `unsigned int` проверяется на этапе компиляции. Индексы и начало массива слотов выравниваются на 64 байта.

При невозможности обмена используется активное ожидание с `cpu_relax`: на x86 выполняется инструкция `pause`. Неблокирующий `waitpid` у родителя и повторный `getppid` у ребёнка вызываются только после 1024 неуспешных попыток обмена. Флаг отмены проверяется в каждой итерации ожидания. Интервал задан числом попыток, а не фиксированным временем.

Описание программы

Основные модули и их назначение:

- `parent.c`: открытие файла, создание канала shared memory, запуск ребёнка, чередование отправки блоков и чтения результатов, обработка сигналов;
- `child.c`: подключение к общей памяти, обработка запросов, отправка сумм и диагностики переполнения, проверка исчезновения родителя;
- `app/protocol.h`: типы сообщений и полезная нагрузка;
- `app/channel.h`: две очереди и атомарные флаги готовности и отмены;
- `app/spsc.c`, `app/spsc.h`: инициализация очереди `spsc_init`; получение свободного слота и публикация через `spsc_acquire_slot/spsc_commit_slot`; просмотр готового сообщения и освобождение слота через `spsc_peek_slot/spsc_consume_slot`; копирующие обёртки `spsc_try_push/spsc_try_pop`;
- `app/parser.c`: потоковый разбор чисел и суммирование строк;
- `common/os/posix`: POSIX-обёртки для файлов, shared memory и процессов;
- `common/raii.h`: автоматическая очистка и передача владения ресурсами;

Тип сообщения	Назначение
`MESSAGE_DATA`	Блок входного файла и число действительных байт.
`MESSAGE_END`	Конец входных данных.
`MESSAGE_RESULT`	Сумма, число слагаемых и номер строки.
`MESSAGE_OVERFLOW`	Переполнение с указанием номера строки.
`MESSAGE_DONE`	Подтверждение завершения обработки.

Структура `parser_t` хранит модуль текущего числа, знак, признаки наличия знака и цифр, сумму, количество чисел и номер строки. Состояние сохраняется между блоками.

parser_feed обрабатывает символ, parser_flush завершает число, parser_take_line формирует результат и сбрасывает строку. Пустые строки пропускаются; EOF обрабатывается и без завершающего перевода строки. Границы int, промежуточная сумма и счётчик чисел проверяются до переполнения. В соответствии с предыдущей реализацией прочие символы разделяют числа, одиночные знаки игнорируются.

Объект shared memory создаётся эксклюзивно с правами 0600. После ftruncate выполняется mmap; файловый дескриптор затем можно закрыть. Ребёнок открывает тот же объект и проверяет ожидаемый размер. После подтверждения child_ready родитель вызывает shm_unlink: имя удаляется, а существующие отображения продолжают работать до munmap. В shared memory хранятся значения и индексы без указателей на локальную память.

Как и в предыдущей работе, внутри обёртки запуска остаётся служебный pipe с FD_CLOEXEC. Он передаёт только errno при ошибке execv. Успешный exec закрывает дескриптор, и родитель получает EOF. Числа и результаты вычислений через этот канал не передаются. После неудачного exec используется _exit, чтобы не вызвать функции очистки, унаследованные от родителя.

RAII-подобная очистка реализована атрибутом cleanup GCC/Clang. Переменные raii(os_file_handle) и raii(os_mapped_shm) освобождают ресурс при выходе из области видимости, включая ранний return. Объект ребёнка объявлен позже mapping и очищается раньше: выставляет отмену, при необходимости посылает SIGTERM и вызывает waitpid. После обычного ожидания PID сбрасывается. Операция move возвращает ресурс и очищает исходную обёртку, предотвращая повторное освобождение. При частично неудавшемся создании mapping удаляется уже созданное имя объекта.

Системный интерфейс	Использование
open/read/write/close	Чтение файла, вывод строк, служебный канал.
shm_open/shm_unlink	Создание, открытие и удаление имени общей памяти.
ftruncate/fstat	Задание и проверка размера объекта.
mmap/munmap	Создание и освобождение отображения.
fork/execv	Создание процесса и запуск программы child.
pipe/fcntl	Канал ошибок exec и установка CLOEXEC.
waitpid/getppid/kill	Проверка и завершение процессов.
sigaction	Обработка SIGINT/SIGTERM, игнорирование SIGPIPE.

Результаты

Обе программы собраны GCC в конфигурации Release с оптимизацией -O3 -DNDEBUG. Основная программа собрана в build-release, канальный вариант из проекта os-lab-1 без изменения его исходников - в build-release/lab1. Приведена последняя серия из 22 измерений: по 11 запусков каждой реализации на одном и том же входном файле.

Проверка корректности

Для файла `src/tests/report_sample.txt`

```
10 20 30
-5 7 8
100 -25 5 0
```

получен следующий вывод:

```
Sum of 3 numbers: 60
Sum of 3 numbers: 10
Sum of 4 numbers: 80
```

Вывод обеих реализаций на основном входе совпал; проверены все 100 000 строк результатов.

Условия измерения

Замеры выполнены 12 ноября 2026 года по московскому времени на Intel Core i5-13420H (4 P-ядра с SMT и 4 E-ядра, 12 логических CPU), x86_64. Система: Linux 6.19.14-108.fc42.x86_64, glibc 2.41; GCC 15.2.1, CMake 3.31.11. Родитель и ребёнок наследуют маску CPU {2,4}, соответствующую двум различным физическим P-ядрам. Миграции между этими ядрами возможны.

Перед измерениями выполнено по 3 прогревочных запуска, затем по 11 измеряемых запусков каждой реализации. Файловый кеш прогрет, вывод направлен в `/dev/null`. Фоновая нагрузка и частоты CPU не фиксировались.

Измерялось полное время выполнения программы. В него входят запуск процессов, чтение файла, разбор чисел, IPC, форматирование вывода и очистка ресурсов. Сравниваются полные программы, а не отдельные операции очереди. В lab4 вывод буферизуется и явно сбрасывается в конце работы, тогда как lab1 вызывает `fflush` после каждой строки. Поэтому измеренное различие времени включает влияние буферизации вывода.

Вход	Байты	Числа	Строки-результаты
`big_data.txt`	2 854 756	650 057	100 000

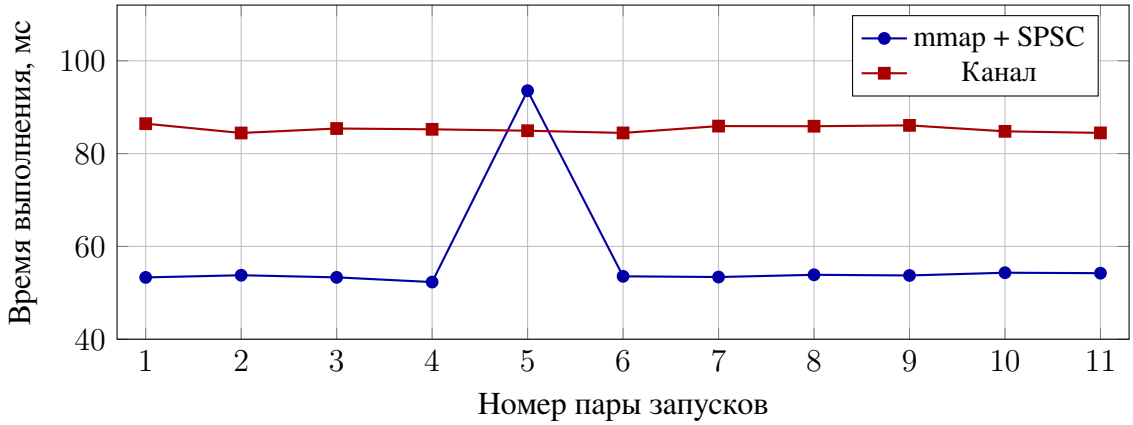
Использован существующий файл `src/tests/big_data.txt` без изменения.

Время выполнения

В таблице приведены показатели по всем 11 измерениям каждой реализации, в миллисекундах. Выбросы не исключались; σ - выборочное стандартное отклонение. Минимум и максимум не являются доверительным интервалом.

Вариант	Медиана	Среднее	σ	Min	Max
mmap + SPSC	53,743	57,225	12,062	52,319	93,556
Канал	85,233	85,281	0,723	84,442	86,448

Медиана mmap составила 53,743 мс, канала - 85,233 мс. В этой серии lab4 быстрее в 1,586 раза, то есть время выполнения меньше на 36,95%. Пропускная способность mmap равна 50,658 МиБ/с, канала - 31,942 МиБ/с.



На пятом запуске mmap наблюдалось время 93,556 мс; этот результат оставлен в выборке. Поэтому среднее и стандартное отклонение mmap заметно отличаются от медианы. По этим измерениям нельзя установить причину выброса.

Показатель	mmap + SPSC	Канал
Медиана суммарного CPU time, мс	101,011	147,253
Медиана переключений контекста	6	328

CPU time и переключения получены из разности `getrusage(RUSAGE_CHILDREN)` до и после запуска. В Linux учитываются и потомки, которых дождался измеряемый родитель. Суммарное CPU time может превышать время выполнения при работе процессов одновременно на двух ядрах.

Текущая очередь содержит 256 слотов, а ребёнок читает вход через `peek/consume` без копирования блока в локальную структуру. Ответы пока передаются копирующими обёртками: размер `message_t` равен 4112 байт из-за общего union с входным блоком. Разделение типов входных блоков и коротких ответов может уменьшить эту работу; его эффект требует отдельного измерения. Результаты данной серии относятся к текущим полным программам и не доказывают превосходство одного механизма IPC во всех условиях.

Выводы

Реализован двусторонний обмен между двумя отдельными программами через один объект POSIX shared memory. Ребёнок создаётся с помощью fork/exec; прикладные запросы и ответы передаются двумя SPSC-очередями. Порядок acquire/release позволяет публиковать сообщения без мьютекса. Интерфейсы acquire/commit и peek/consume устраняют промежуточные копии входных блоков в пользовательской памяти у обоих процессов.

Корректность обмена требует согласованного протокола: родитель должен вычитывать ответы, EOF передаётся явным сообщением, окончание обработки подтверждается DONE. RAII-подобные обёртки связывают ресурсы с областью видимости и обеспечивают правильный порядок очистки.

На файле из 100 000 строк медиана времени lab4 равна 53,743 мс, lab1 - 85,233 мс: ускорение составило 1,586 раза. Сравнение включает всю обработку и различия буферизации вывода. Активное ожидание расходует CPU, а короткие ответы пока копируются как крупные структуры. Для дальнейшего исследования можно разделить типы запросов и ответов и отдельно измерить эффект этого изменения при одинаковых остальных параметрах.

Исходная программа

Ниже приведён снимок исходников измеренной версии. Пути указаны относительно каталога src. Листинги включены непосредственно в этот LaTeX-файл, поэтому для сборки отчёта не требуется доступ к исходному дереву проекта.

parent.c

```c
#include "app/channel.h"
#include "common/os/os.h"
#include "common/os/posix/file/raii.h"
#include "common/os/posix/mapped_shm/mapped_shm_raii.h"
#include "common/spin.h"

#include <errno.h>
#include <inttypes.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    channel_t *channel;
    os_process_handle_t process;
} child_process_t;

static void child_process_cleanup(child_process_t *child) {
    int saved_errno = errno;
    atomic_store_explicit(&child->channel->cancelled, 1, memory_order_release);
    if (child->process != OS_INVALID_HANDLE) {
        int exit_code;
        os_process_terminate(child->process);
        (void)os_process_wait(child->process, &exit_code);
    }
    errno = saved_errno;
}

static bool child_process_valid(const child_process_t *child) {
    return child != NULL && child->channel != NULL;
}

static void child_process_reset(child_process_t *child) {
    *child = (child_process_t){.process = OS_INVALID_HANDLE};
}

DEFINE_RAII_TYPE(child_process, child_process_t, child_process_cleanup,
                 child_process_valid, child_process_reset)

static volatile sig_atomic_t interrupted;

static void on_signal(int signum) {
    (void)signum;
    interrupted = 1;
}

static int install_signals(void) {
    struct sigaction action = {0};
    sigemptyset(&action.sa_mask);
    action.sa_handler = on_signal;
    if (sigaction(SIGINT, &action, NULL) < 0 ||
        sigaction(SIGTERM, &action, NULL) < 0) return -1;
    action.sa_handler = SIG_IGN;
    return sigaction(SIGPIPE, &action, NULL);
}

static char *read_path(const char *prompt) {
    char *line = NULL;
    size_t capacity = 0;
    printf("%s", prompt);
```

```
    fflush(stdout);
    if (getline(&line, &capacity, stdin) < 0) {
        free(line);
        return NULL;
    }
    line[strcspn(line, "\r\n")] = '\0';
    return line;
}

static os_mapped_shm_t create_channel(void) {
    os_shm_info_t info = {
        .flags = OS_O_RDWR | OS_O_CREAT | OS_O_EXCL,
        .mode = OS_S_IRUSR | OS_S_IWUSR,
    };
    for (unsigned attempt = 0; attempt < 1024; ++attempt) {
        snprintf(info.name, sizeof(info.name), "/os_lab_4_%jd_%u",
                 (intmax_t)os_process_id(), attempt);
        os_mapped_shm_t shm = os_mapped_shm_create(
            &info, sizeof(channel_t), OS_PROT_READ | OS_PROT_WRITE, OS_MAP_SHARED);
        if (shm.mmap_addr != NULL || errno != EEXIST) return shm;
    }
    errno = EEXIST;
    return (os_mapped_shm_t){.fd = OS_INVALID_HANDLE};
}

static bool handle_response(const message_t *message, bool *done) {
    switch (message->kind) {
    case MESSAGE_RESULT:
        if (printf("Sum of %d numbers: %d\n", message->payload.result.count,
                   message->payload.result.sum) < 0) {
            perror("Failed to write result");
            return false;
        }
        return true;
    case MESSAGE_OVERFLOW:
        fprintf(stderr, "Integer overflow in line %zu while parsing or summing\n",
                message->payload.result.line);
        return false;
    case MESSAGE_DONE:
        *done = true;
        return true;
    default:
        fprintf(stderr, "Unexpected child message: %d\n", (int)message->kind);
        return false;
    }
}

static int run(const char *filename, const char *child_path) {
    raii(os_file_handle) file = os_file_open(filename, OS_FILE_READ);
    if (file == OS_INVALID_HANDLE) {
        perror("Failed to open input file");
        return EXIT_FAILURE;
    }
    raii(os_mapped_shm) shm = create_channel();
    if (shm.mmap_addr == NULL) {
        perror("Failed to create shared memory");
        return EXIT_FAILURE;
    }
    channel_t *channel = shm.mmap_addr;
    channel_init(channel);

    char parent_pid[32];
    snprintf(parent_pid, sizeof(parent_pid), "%jd", (intmax_t)os_process_id());
    char *child_argv[] = {(char *)child_path, shm.name, parent_pid, NULL};
    raii(child_process) child = {
        .channel = channel,
        .process = os_process_spawn(child_path, child_argv),
    };
    if (child.process == OS_INVALID_HANDLE) {
        perror("Failed to start child");
        return EXIT_FAILURE;
    }

```

```
    int child_exit = EXIT_FAILURE;
    bool eof_sent = false;
    bool done = false;
    unsigned idle_spins = 0;
    while (!done) {
        if (interrupted) {
            fprintf(stderr, "Interrupted\n");
            return EXIT_FAILURE;
        }
        if (shm.is_owner && atomic_load_explicit(&channel->child_ready,
                                                 memory_order_acquire)) {
            if (os_shm_unlink(shm.name) < 0) {
                perror("Failed to unlink shared memory");
                return EXIT_FAILURE;
            }
            shm.is_owner = false;
        }
        bool progress = false;
        message_t response;
        for (unsigned i = 0; i < SPSC_SLOT_COUNT; ++i) {
            if (!spsc_try_pop(&channel->responses, &response)) break;
            progress = true;
            if (!handle_response(&response, &done)) return EXIT_FAILURE;
            if (done) break;
        }
        if (done) break;
        if (child.process == OS_INVALID_HANDLE) {
            fprintf(stderr, "Child exited without completion (status %d)\n", child_exit);
            return EXIT_FAILURE;
        }

        if (!eof_sent) {
            message_t *slot = spsc_acquire_slot(&channel->requests);
            if (slot != NULL) {
                os_ssize_t bytes = os_file_read(file, slot->payload.data.bytes,
                                                INPUT_BLOCK_SIZE);
                if (bytes < 0) {
                    perror("Failed to read input file");
                    return EXIT_FAILURE;
                }
                slot->kind = bytes == 0 ? MESSAGE_END : MESSAGE_DATA;
                slot->payload.data.size = (size_t)bytes;
                spsc_commit_slot(&channel->requests);
                eof_sent = bytes == 0;
                progress = true;
            }
        }
        if (!progress) {
            cpu_relax();
            if (++idle_spins < PEER_POLL_SPINS) continue;
            idle_spins = 0;
            int status = os_process_poll(child.process, &child_exit);
            if (status < 0) {
                perror("Failed to poll child");
                return EXIT_FAILURE;
            }
            if (status > 0) {
                child.process = OS_INVALID_HANDLE;
            }
        } else {
            idle_spins = 0;
        }
    }

    if (!eof_sent) {
        fprintf(stderr, "Child completed before receiving end of input\n");
        return EXIT_FAILURE;
    }
    if (child.process != OS_INVALID_HANDLE) {
        if (os_process_wait(child.process, &child_exit) < 0) {
            perror("Failed to wait for child");
            return EXIT_FAILURE;
        }
```

```
        child.process = OS_INVALID_HANDLE;
    }
    if (child_exit != EXIT_SUCCESS) {
        fprintf(stderr, "Child failed (status %d)\n", child_exit);
        return EXIT_FAILURE;
    }
    if ( fflush(stdout) == EOF) {
        perror("Failed to write to standard output");
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}

int main(int argc, char *argv[]) {
    if (argc != 1 && argc != 3) {
        fprintf(stderr, "Usage: %s [data_file child_executable]\n", argv[0]);
        return EXIT_FAILURE;
    }
    if (install_signals() < 0) {
        perror("Failed to install signal handlers");
        return EXIT_FAILURE;
    }
    if (argc == 3) return run(argv[1], argv[2]);

    char *filename = read_path("Enter filename: ");
    char *child_path = filename != NULL ? read_path("Enter path to the child process: ") : NULL;
    int result = EXIT_FAILURE;
    if (filename == NULL || child_path == NULL) {
        fprintf(stderr, "Failed to read paths from stdin\n");
    }
    else {
        result = run(filename, child_path);
    }
    free(filename);
    free(child_path);
    return result;
}
```

child.c

```
#include "app/channel.h"
#include "app/parser.h"
#include "common/os/os.h"
#include "common/os/posix/mapped_shm/mapped_shm_raii.h"
#include "common/spin.h"

#include <errno.h>
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    os_process_handle_t parent;
    unsigned idle_spins;
} parent_watch_t;

static bool parent_disappeared(parent_watch_t *watch) {
    if (++watch->idle_spins < PEER_POLL_SPINS) return false;
    watch->idle_spins = 0;
    return os_process_parent_id() != watch->parent;
}

static bool send_response(channel_t *channel, parent_watch_t *watch,
                          const message_t *message) {
    while (!atomic_load_explicit(&channel->cancelled, memory_order_acquire)) {
        if (spsc_try_push(&channel->responses, message)) return true;
        if (parent_disappeared(watch)) return false;
```

```
        cpu_relax();
    }
    return false;
}

static int process_input(channel_t *channel, os_process_handle_t parent) {
    parser_t parser;
    parser_init(&parser);
    parent_watch_t watch = {.parent = parent};
    while (!atomic_load_explicit(&channel->cancelled, memory_order_acquire)) {
        const message_t *request = spsc_peek_slot(&channel->requests);

        if (request == NULL) {
            if (parent_disappeared(&watch)) return EXIT_FAILURE;
            cpu_relax();
            continue;
        }
        message_t response;
        if (request->kind == MESSAGE_END) {
            spsc_consume_slot(&channel->requests);
            if (!parser_flush(&parser)) {
                message_t error = {
                    .kind = MESSAGE_OVERFLOW,
                    .payload.result = {.line = parser.line},
                };
                (void)send_response(channel, &watch, &error);
                return EXIT_FAILURE;
            }
            if (parser_take_line(&parser, &response) &&
                !send_response(channel, &watch, &response)) return EXIT_FAILURE;
            response = (message_t){.kind = MESSAGE_DONE};
            return send_response(channel, &watch, &response) ? EXIT_SUCCESS : EXIT_FAILURE;
        }
        if (request->kind != MESSAGE_DATA || request->payload.data.size > INPUT_BLOCK_SIZE) {
            fprintf(stderr, "Invalid parent message\n");
            return EXIT_FAILURE;
        }
        for (size_t i = 0; i < request->payload.data.size; ++i) {
            char ch = request->payload.data.bytes[i];
            if (!parser_feed(&parser, ch)) {
                message_t error = {
                    .kind = MESSAGE_OVERFLOW,
                    .payload.result = {.line = parser.line},
                };
                (void)send_response(channel, &watch, &error);
                return EXIT_FAILURE;
            }
            if (ch == '\n' && parser_take_line(&parser, &response) &&
                !send_response(channel, &watch, &response)) return EXIT_FAILURE;
        }
        spsc_consume_slot(&channel->requests);

    }
    return EXIT_FAILURE;
}

int main(int argc, char *argv[]) {
    if (argc != 3 || strlen(argv[1]) >= SHM_NAME_LEN) {
        fprintf(stderr, "Usage: %s shm_name parent_pid (started by parent)\n", argv[0]);
        return EXIT_FAILURE;
    }
    char *end;
    errno = 0;
    intmax_t pid = strtoimax(argv[2], &end, 10);
    if (errno != 0 || *end != '\0' || pid <= 0 || pid > INTPTR_MAX ||
        os_process_parent_id() != (os_process_handle_t)pid) {
        fprintf(stderr, "Invalid parent PID\n");
        return EXIT_FAILURE;
    }
    os_shm_info_t info = {.flags = OS_O_RDWR};
    memcpy(info.name, argv[1], strlen(argv[1]) + 1);
    raii(os_mapped_shm) shm = os_mapped_shm_create(
        &info, sizeof(channel_t), OS_PROT_READ | OS_PROT_WRITE, OS_MAP_SHARED);
```

```
    if (shm.mmap_addr == NULL) {
        perror("Failed to open shared memory");
        return EXIT_FAILURE;
    }
    channel_t *channel = shm.mmap_addr;
    atomic_store_explicit(&channel->child_ready, 1, memory_order_release);
    return process_input(channel, (os_process_handle_t)pid);
}
```

app/protocol.h

```
#ifndef APP_PROTOCOL_H
#define APP_PROTOCOL_H

#include <stddef.h>

#define INPUT_BLOCK_SIZE 4096

typedef enum {
    MESSAGE_DATA,
    MESSAGE_END,
    MESSAGE_RESULT,
    MESSAGE_OVERFLOW,
    MESSAGE_DONE
} message_kind_t;

typedef struct {
    message_kind_t kind;
    union {
        struct {
            size_t size;
            char bytes[INPUT_BLOCK_SIZE];
        } data;
        struct {
            int sum;
            int count;
            size_t line;
        } result;
    } payload;
} message_t;

#endif
```

app/channel.h

```
#ifndef APP_CHANNEL_H
#define APP_CHANNEL_H

#include "app/spsc.h"

#define PEER_POLL_SPINS 1024U

typedef struct {
    spsc_queue_t requests;
    spsc_queue_t responses;
    atomic_uint cancelled;
    atomic_uint child_ready;
} channel_t;

static inline void channel_init(channel_t *channel) {
    spsc_init(&channel->requests);
    spsc_init(&channel->responses);
    atomic_init(&channel->cancelled, 0);
    atomic_init(&channel->child_ready, 0);
}

```

```
#endif
```

app/spsc.h

```
#ifndef APP_SPSC_H
#define APP_SPSC_H

#include <stdbool.h>
#include <stdatomic.h>

#include "app/protocol.h"

#define SPSC_SLOT_COUNT 256U
#define SPSC_CACHE_LINE_SIZE 64U

_Static_assert(ATOMIC_INT_LOCK_FREE == 2, "Always lock-free unsigned atomics required");
_Static_assert((SPSC_SLOT_COUNT & (SPSC_SLOT_COUNT - 1U)) == 0U && SPSC_SLOT_COUNT >= 2U,
               "SPSC_SLOT_COUNT must be a power of two >= 2");


typedef struct {
    _Alignas(SPSC_CACHE_LINE_SIZE) atomic_uint read_index;
    _Alignas(SPSC_CACHE_LINE_SIZE) atomic_uint write_index;
    _Alignas(SPSC_CACHE_LINE_SIZE) message_t slots[SPSC_SLOT_COUNT];
} spsc_queue_t;

void spsc_init(spsc_queue_t *queue);

message_t *spsc_acquire_slot(spsc_queue_t *queue);
void spsc_commit_slot(spsc_queue_t *queue);
bool spsc_try_push(spsc_queue_t *queue, const message_t *message);

bool spsc_try_pop(spsc_queue_t *queue, message_t *message);
const message_t *spsc_peek_slot(const spsc_queue_t *queue);
void spsc_consume_slot(spsc_queue_t *queue);

#endif
```

app/spsc.c

```
#include "app/spsc.h"

void spsc_init(spsc_queue_t *queue) {
    atomic_init(&queue->read_index, 0);
    atomic_init(&queue->write_index, 0);
}

message_t *spsc_acquire_slot(spsc_queue_t *queue) {
    unsigned write = atomic_load_explicit(&queue->write_index, memory_order_relaxed);
    unsigned next = (write + 1U) % SPSC_SLOT_COUNT;
    if (next == atomic_load_explicit(&queue->read_index, memory_order_acquire)) {
        return NULL;
    }
    return &queue->slots[write];
}

void spsc_commit_slot(spsc_queue_t *queue) {
    unsigned write = atomic_load_explicit(&queue->write_index, memory_order_relaxed);
    atomic_store_explicit(&queue->write_index, (write + 1U) % SPSC_SLOT_COUNT,
                          memory_order_release);
}

bool spsc_try_push(spsc_queue_t *queue, const message_t *message) {
    message_t *slot = spsc_acquire_slot(queue);
    if (slot == NULL) return false;
    *slot = *message;
```

```
    spsc_commit_slot(queue);
    return true;
}

const message_t *spsc_peek_slot(const spsc_queue_t *queue) {
    unsigned read = atomic_load_explicit(&queue->read_index, memory_order_relaxed);
    if (read == atomic_load_explicit(&queue->write_index, memory_order_acquire)) {
        return NULL;
    }
    return &queue->slots[read];
}

void spsc_consume_slot(spsc_queue_t *queue) {
    unsigned read = atomic_load_explicit(&queue->read_index, memory_order_relaxed);
    atomic_store_explicit(&queue->read_index, (read + 1U) % SPSC_SLOT_COUNT,
                          memory_order_release);
}

bool spsc_try_pop(spsc_queue_t *queue, message_t *message) {
    const message_t *slot = spsc_peek_slot(queue);
    if (slot == NULL) return false;
    *message = *slot;
    spsc_consume_slot(queue);
    return true;
}
```

app/parser.h

```
#ifndef APP_PARSER_H
#define APP_PARSER_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include "app/protocol.h"

typedef struct {
    int64_t value;
    int64_t sum;
    int sign;
    int count;
    bool has_digits;
    bool has_sign;
    size_t line;
} parser_t;

void parser_init(parser_t *parser);
bool parser_feed(parser_t *parser, char ch);
bool parser_flush(parser_t *parser);
bool parser_take_line(parser_t *parser, message_t *result);

#endif
```

app/parser.c

```
#include "app/parser.h"

#include <ctype.h>
#include <limits.h>

static void reset_number(parser_t *parser) {
    parser->value = 0;
    parser->sign = 1;
    parser->has_digits = false;
    parser->has_sign = false;
```

```
}

void parser_init(parser_t *parser) {
    *parser = (parser_t){.sign = 1, .line = 1};
}

bool parser_flush(parser_t *parser) {
    if (!parser->has_digits) {
        reset_number(parser);
        return true;
    }
    int64_t number = parser->value * parser->sign;
    if ((number > 0 && parser->sum > INT_MAX - number) ||
        (number < 0 && parser->sum < INT_MIN - number) ||
        parser->count == INT_MAX)
        return false;
    parser->sum += number;
    ++parser->count;
    reset_number(parser);
    return true;
}

bool parser_feed(parser_t *parser, char ch) {
    if (isspace((unsigned char)ch)) return parser_flush(parser);
    if ((ch == '-' || ch == '+') && !parser->has_digits && !parser->has_sign) {
        parser->sign = ch == '-' ? -1 : 1;
        parser->has_sign = true;
        return true;
    }
    if (isdigit((unsigned char)ch)) {
        int digit = ch - '0';
        int64_t limit = parser->sign < 0 ? -(int64_t)INT_MIN : INT_MAX;
        if (parser->value > (limit - digit) / 10) return false;
        parser->value = parser->value * 10 + digit;
        parser->has_digits = true;
        return true;
    }
    return parser_flush(parser);
}

bool parser_take_line(parser_t *parser, message_t *result) {
    bool present = parser->count != 0;
    *result = (message_t){
        .kind = MESSAGE_RESULT,
        .payload.result = {.sum = (int)parser->sum, .count = parser->count,
                           .line = parser->line},
    };
    reset_number(parser);
    parser->sum = 0;
    parser->count = 0;
    ++parser->line;
    return present;
}
```

common/raii.h

```
#ifndef RAII_H
#define RAII_H

#include "common/unlikely.h"

#if defined(__clang__) || defined(__GNUC__)

#define RAII_DETAIL_CAT_IMPL(left, right) left##right
#define RAII_DETAIL_CAT(left, right) RAII_DETAIL_CAT_IMPL(left, right)
#define RAII_DETAIL_SYMBOL(TypeName, suffix) \
RAII_DETAIL_CAT(RAII_DETAIL_CAT(raii_detail_, TypeName), suffix)


```

```
#define DEFINE_RAII_TYPE(TypeName, BaseType, CleanupFunc, IsValidFunc, ResetFunc) \
    typedef BaseType TypeName;\
    static inline void RAII_DETAIL_SYMBOL(TypeName, _cleanup)(TypeName *ptr) { \
        if (unlikely(!ptr)) return; \
        if (likely(IsValidFunc(ptr))) { \
            CleanupFunc(ptr); \
            ResetFunc(ptr); \
        } \
    }

#define DEFINE_RAII_TRIVIAL_MOVE(TypeName, IsValidFunc, ResetFunc) \
    static inline TypeName RAII_DETAIL_SYMBOL(TypeName, _take)(TypeName *source) { \
        TypeName val = {0}; \
        ResetFunc(&val); \
        if (likely(source && IsValidFunc(source))) { \
            val = *source; \
            ResetFunc(source); \
        } \
        return val; \
    } \
    static inline void RAII_DETAIL_SYMBOL(TypeName, _move_assign)( \
            TypeName *destination, TypeName *source) { \
        if (unlikely(!destination || destination == source)) return; \
        RAII_DETAIL_SYMBOL(TypeName, _cleanup)(destination); \
        if (likely(source && IsValidFunc(source))) { \
            *destination = *source; \
            ResetFunc(source); \
        } else { \
            ResetFunc(destination); \
        } \
    }


#define raii(TypeName) TypeName __attribute__((cleanup(RAII_DETAIL_SYMBOL(TypeName, _cleanup))))

#define move(TypeName, var) RAII_DETAIL_SYMBOL(TypeName, _take)(&(var))


#else
#error "This project requires GCC or Clang for RAII support"

#endif

#endif //RAII_H
```

common/spin.h

```
#ifndef COMMON_SPIN_H
#define COMMON_SPIN_H

#if defined(__i386__) || defined(__x86_64__)
#include <immintrin.h>
#else
#include <stdatomic.h>
#endif

static void cpu_relax(void) {
#if defined(__i386__) || defined(__x86_64__)
    _mm_pause();
#elif defined(__aarch64__) || (defined(__arm__) && __ARM_ARCH >= 7)
    __asm__ __volatile__("yield");
#else
    atomic_signal_fence(memory_order_seq_cst);
#endif
}

#endif
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(os_lab_4 VERSION 1.0 LANGUAGES C)

if(NOT UNIX)
    message(FATAL_ERROR "os_lab_4 requires POSIX shared memory and processes")
endif()

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

add_library(lab_options INTERFACE)
target_compile_definitions(lab_options INTERFACE _POSIX_C_SOURCE=200809L)
target_compile_options(lab_options INTERFACE
    $<$<C_COMPILER_ID:GNU,Clang,AppleClang>:-Wall;-Wextra;-Wpedantic>)

add_subdirectory(common)
add_subdirectory(app)

add_executable(parent parent.c)
add_executable(child child.c)
target_link_libraries(parent PRIVATE lab_logic os_lib lab_options)
target_link_libraries(child PRIVATE lab_logic os_lib lab_options)
set_target_properties(parent child PROPERTIES
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_BINARY_DIR}/bin")
```

app/CMakeLists.txt

```
add_library(lab_logic STATIC spsc.c parser.c)
target_include_directories(lab_logic PUBLIC "${PROJECT_SOURCE_DIR}")
target_link_libraries(lab_logic PRIVATE lab_options)
```

common/CMakeLists.txt

```
add_library(os_lib STATIC
    os/posix/file/file.c
    os/posix/mmap.c
    os/posix/shm.c
    os/posix/mapped_shm/mapped_shm.c
    os/posix/process.c)
target_include_directories(os_lib PUBLIC "${PROJECT_SOURCE_DIR}")
target_link_libraries(os_lib PRIVATE lab_options)

# Older POSIX systems provide shm_open in librt rather than libc.
include(CheckSymbolExists)
check_symbol_exists(shm_open "sys/mman.h" HAVE_SHM_OPEN_IN_LIBC)
if(NOT HAVE_SHM_OPEN_IN_LIBC)
    find_library(RT_LIBRARY rt)
    if(NOT RT_LIBRARY)
        message(FATAL_ERROR "POSIX shm_open is unavailable (including librt)")
    endif()
    target_link_libraries(os_lib PRIVATE "${RT_LIBRARY}")
endif()
```

common/os/common.h

```c
#ifndef COMMON_H
#define COMMON_H
#include <stddef.h>
#include <stdint.h>

#define OS_INVALID_HANDLE (-1)

typedef int64_t os_offset_t;
typedef intptr_t os_file_handle_t;
typedef intptr_t os_process_handle_t;
typedef ptrdiff_t os_ssize_t;

#endif //COMMON_H
```

common/os/file.h

```c
#ifndef FILE_H
#define FILE_H

#include "common/os/common.h"

typedef enum {
    OS_FILE_READ = 1,
    OS_FILE_WRITE = 1 << 1,
    OS_FILE_CREATE = 1 << 2,
    OS_FILE_TRUNCATE = 1 << 3,
} os_file_flags_t;

os_file_handle_t os_file_open(const char *path, os_file_flags_t flags);
void os_file_close(os_file_handle_t file);

os_ssize_t os_file_read(os_file_handle_t file, void *buf, size_t count);
os_ssize_t os_file_read_full(os_file_handle_t file, void *buf, size_t count);

os_ssize_t os_file_write(os_file_handle_t file, const void *buf, size_t count);
os_ssize_t os_file_write_full(os_file_handle_t file, const void *buf, size_t count);
int os_file_fsync(os_file_handle_t file);

#endif //FILE_H
```

common/os/mapped_shm.h

```c
#ifndef MAPPED_SHM_H
#define MAPPED_SHM_H

#include "common/os/mmap.h"
#include "common/os/shm.h"

typedef struct {
    char name[SHM_NAME_LEN];
    os_o_flags_t flags;
    os_mode_t mode;
} os_shm_info_t;

typedef struct {
    os_shm_file_t fd;
    size_t size;
    char name[SHM_NAME_LEN];
    bool is_owner;
    void *mmap_addr;
} os_mapped_shm_t;

```

```
os_mapped_shm_t os_mapped_shm_create(os_shm_info_t *shm_info, size_t len, os_map_prot_t prot,
    os_map_flags_t flags);
void os_mapped_shm_destroy(os_mapped_shm_t *shm);

#endif //MAPPED_SHM_H
```

common/os/mmap.h

```
#ifndef MMAP_H
#define MMAP_H

#include "common/os/common.h"

typedef enum {
    OS_PROT_EXEC = 1,
    OS_PROT_READ = 1 << 1,
    OS_PROT_WRITE = 1 << 2,
    OS_PROT_NONE = 1 << 3,
} os_map_prot_t;

typedef enum {
    OS_MAP_SHARED = 1,
    OS_MAP_PRIVATE = 1 << 1,
    OS_MAP_FIXED = 1 << 3,
} os_map_flags_t;

void *os_mmap(void *addr,
    size_t len,
    os_map_prot_t prot,
    os_map_flags_t flags,
    os_file_handle_t fd,
    os_offset_t offset);

int os_munmap(void *addr, size_t len);

#endif //MMAP_H
```

common/os/os.h

```
#ifndef OS_H
#define OS_H

#include "common/os/common.h"
#include "common/os/file.h"
#include "common/os/shm.h"
#include "common/os/mmap.h"
#include "common/os/mapped_shm.h"
#include "common/os/process.h"

#endif //OS_H
```

common/os/posix/container_of.h

```
#ifndef POSIX_CONTAINER_OF_H
#define POSIX_CONTAINER_OF_H

#include <stddef.h>

#if defined(__GNUC__) || defined(__clang__)

#define container_of(ptr, stct, field) ({ \
    const __typeof__( ((stct*)0)->field) *__mptr = (ptr); \
```

```
    (stct*) ((char*) __mptr - offsetof(stct, field)); \
})

#else
#error "This file requires GCC or Clang"
#endif

#endif //POSIX_CONTAINER_OF_H
```

common/os/posix/file/file.c

```
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>

#include "common/os/os.h"
#include "common/unlikely.h"

os_file_handle_t os_file_open(const char *path, os_file_flags_t flags) {
    if (unlikely(!path)) {
        errno = EINVAL;
        return OS_INVALID_HANDLE;
    }
    int posix_flag = O_CLOEXEC;
    if ((flags & OS_FILE_READ) && (flags & OS_FILE_WRITE)) {
        posix_flag |= O_RDWR;
    } else if (flags & OS_FILE_WRITE) {
        posix_flag |= O_WRONLY;
    } else {
        posix_flag |= O_RDONLY;
    }

    if (flags & OS_FILE_CREATE) posix_flag |= O_CREAT;
    if (flags & OS_FILE_TRUNCATE) posix_flag |= O_TRUNC;

    int fd = open(path, posix_flag, 0644);
    if (fd < 0) {
        return OS_INVALID_HANDLE;
    }
    return fd;
}

void os_file_close(os_file_handle_t fd) {
    if (likely(fd != OS_INVALID_HANDLE)) {
        int e = errno;
        close((int)fd);
        errno = e;
    }
}

os_ssize_t os_file_read(os_file_handle_t file, void *buf, size_t count) {
    if (unlikely(file == OS_INVALID_HANDLE)) {
        errno = EINVAL;
        return -1;
    }
    int fd = (int)file;
    os_ssize_t bytes_read;
    do {
        bytes_read = read(fd, buf, count);
    } while (bytes_read < 0 && errno == EINTR);
    return bytes_read;
}

os_ssize_t os_file_read_full(os_file_handle_t file, void *buf, size_t count) {
    if (unlikely(file == OS_INVALID_HANDLE)) {
        errno = EINVAL;
        return -1;
    }
    int fd = (int)file;
```

```
    os_ssize_t total_read = 0;
    size_t remaining = count;
    char* ptr = (char*)buf;
    while (remaining > 0) {
        ssize_t bytes_read = read(fd, ptr, remaining);
        if (bytes_read < 0) {
            if (errno == EINTR) continue;
            return total_read > 0 ? total_read : -1;
        }
        if (bytes_read == 0) break;
        ptr += bytes_read;
        remaining -= bytes_read;
        total_read += bytes_read;
    }
    return total_read;
}

os_ssize_t os_file_write(os_file_handle_t file, const void *buf, size_t count) {
    if (unlikely(file == OS_INVALID_HANDLE)) {
        errno = EINVAL;
        return -1;
    }
    int fd = (int)file;
    os_ssize_t bytes_written;
    do {
        bytes_written = write(fd, buf, count);
    } while (bytes_written < 0 && errno == EINTR);
    return bytes_written;
}

os_ssize_t os_file_write_full(os_file_handle_t file, const void *buf, size_t count) {
    if (unlikely(file == OS_INVALID_HANDLE)) {
        errno = EINVAL;
        return -1;
    }

    int fd = (int)file;
    os_ssize_t total_written = 0;
    size_t remaining = count;
    const char* ptr = buf;
    while (remaining > 0) {
        ssize_t bytes_written = write(fd, ptr, remaining);
        if (bytes_written < 0) {
            if (errno == EINTR) continue;
            return total_written > 0 ? total_written : -1;
        }
        if (bytes_written == 0) {
            errno = EIO;
            return total_written > 0 ? total_written : -1;
        }
        total_written += bytes_written;
        ptr += bytes_written;
        remaining -= bytes_written;
    }
    return total_written;
}

int os_file_fsync(os_file_handle_t file) {
    if (unlikely(file == OS_INVALID_HANDLE)) {
        errno = EINVAL;
        return -1;
    }
    int fd = (int)file;
    int res;
    while ((res = fsync(fd)) < 0 && errno == EINTR) {}
    return res;
}
```

common/os/posix/file/raii.h

```c
#ifndef FILE_RAII_H
#define FILE_RAII_H

#include <errno.h>
#include <stdbool.h>
#include <unistd.h>
#include <stddef.h>

#include "common/os/file.h"
#include "common/unlikely.h"
#include "common/raii.h"


static void os_file_cleanup(os_file_handle_t *fd) {
    (void)os_file_close(*fd);
}

static bool os_file_valid(const os_file_handle_t *fd) {
    return fd && *fd != OS_INVALID_HANDLE;
}

static void os_file_reset(os_file_handle_t *fd) {
    if (unlikely(fd == NULL || *fd == OS_INVALID_HANDLE)) return;
    *fd = OS_INVALID_HANDLE;
}

DEFINE_RAII_TYPE(os_file_handle, os_file_handle_t, os_file_cleanup, os_file_valid, os_file_reset)
DEFINE_RAII_TRIVIAL_MOVE(os_file_handle, os_file_valid, os_file_reset)

#endif //FILE_RAII_H
```

common/os/posix/mapped_shm/mapped_shm.c

```c
#include "common/os/mapped_shm.h"
#include "common/os/posix/file/raii.h"
#include "common/os/posix/mapped_shm/mapped_shm_raii.h"

#include <errno.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

os_mapped_shm_t os_mapped_shm_create(os_shm_info_t *info, size_t len,
                                     os_map_prot_t prot, os_map_flags_t flags) {
    const os_mapped_shm_t invalid = {.fd = OS_INVALID_HANDLE};
    if (info == NULL || len == 0 || info->name[0] != '/' ||
        strnlen(info->name, SHM_NAME_LEN) >= SHM_NAME_LEN) {
        errno = EINVAL;
        return invalid;
    }

    /* Creation is exclusive: never truncate or unlink another owner's object. */
    bool owner = (info->flags & OS_O_CREAT) != 0;
    os_o_flags_t open_flags = info->flags;
    if (owner) open_flags |= OS_O_EXCL;
    else if (open_flags & OS_O_TRUNC) {
        errno = EINVAL;
        return invalid;
    }
    raii(os_file_handle) fd = os_shm_open(info->name, open_flags, info->mode);
    if (fd == OS_INVALID_HANDLE) return invalid;

    raii(os_mapped_shm) result = {
        .fd = OS_INVALID_HANDLE,
        .size = len,
```

```
        .is_owner = owner,
    };
    memcpy(result.name, info->name, strlen(info->name) + 1);
    if (owner) {
        if (ftruncate((int)fd, (off_t)len) < 0) return invalid;
    } else {
        struct stat st;
        if (fstat((int)fd, &st) < 0) return invalid;
        if (st.st_size < 0 || (uintmax_t)st.st_size != (uintmax_t)len) {
            errno = EINVAL;
            return invalid;
        }
    }
    void *addr = os_mmap(NULL, len, prot, flags, fd, 0);
    if (addr == MAP_FAILED) return invalid;
    result.mmap_addr = addr;
    return move(os_mapped_shm, result);
}

void os_mapped_shm_destroy(os_mapped_shm_t *shm) {
    if (shm == NULL) return;
    int saved_errno = errno;
    if (shm->mmap_addr != NULL && shm->size > 0)
        os_munmap(shm->mmap_addr, shm->size);
    if (shm->is_owner && shm->name[0] != '\0') os_shm_unlink(shm->name);
    *shm = (os_mapped_shm_t){.fd = OS_INVALID_HANDLE};
    errno = saved_errno;
}
```

common/os/posix/mapped_shm/mapped_shm_raii.h

```
#ifndef MAPPED_SHM_RAII_H
#define MAPPED_SHM_RAII_H

#include "common/os/mapped_shm.h"
#include "common/unlikely.h"
#include "common/raii.h"

static void mapped_shm_cleanup(os_mapped_shm_t *shm) {
    os_mapped_shm_destroy(shm);
}

static bool mapped_shm_valid(const os_mapped_shm_t *shm) {
    return shm != NULL && ((shm->mmap_addr != NULL && shm->size > 0) ||
                           (shm->is_owner && shm->name[0] != '\0'));
}

static void mapped_shm_reset(os_mapped_shm_t *shm) {
    if (unlikely(shm == NULL)) return;
    shm->fd = OS_INVALID_HANDLE;
    shm->name[0] = '\0';
    shm->mmap_addr = NULL;
    shm->size = 0;
    shm->is_owner = false;
}

DEFINE_RAII_TYPE(os_mapped_shm, os_mapped_shm_t, mapped_shm_cleanup, mapped_shm_valid, mapped_shm_reset
    )
DEFINE_RAII_TRIVIAL_MOVE(os_mapped_shm, mapped_shm_valid, mapped_shm_reset)

#endif //MAPPED_SHM_RAII_H
```

common/os/posix/mmap.c

```c
#include "common/os/mmap.h"

#include <sys/mman.h>

static int posix_prot(os_map_prot_t prot) {
    int res = 0;
    if (prot & OS_PROT_EXEC) res |= PROT_EXEC;
    if (prot & OS_PROT_WRITE) res |= PROT_WRITE;
    if (prot & OS_PROT_READ) res |= PROT_READ;
    if (prot & OS_PROT_NONE) res |= PROT_NONE;

    return res;
}

static int posix_flags(os_map_flags_t flags) {
    int res = 0;
    if (flags & OS_MAP_PRIVATE) res |= MAP_PRIVATE;
    if (flags & OS_MAP_SHARED) res |= MAP_SHARED;
    if (flags & OS_MAP_FIXED) res |= MAP_FIXED;
    return res;
}

void *os_mmap(void *addr,
    size_t len,
    os_map_prot_t prot,
    os_map_flags_t flags,
    os_file_handle_t fd,
    os_offset_t offset)
{
    int pprot = posix_prot(prot);
    int pflags = posix_flags(flags);
    return mmap(addr, len, pprot, pflags, (int)fd, offset);
}

int os_munmap(void *addr, size_t len) {
    return munmap(addr, len);
}
```

common/os/posix/process.c

```c
#include "common/os/process.h"
#include "common/os/posix/file/raii.h"

#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

os_process_handle_t os_process_spawn(const char *path, char *const argv[]) {
    if (unlikely(path == NULL || argv == NULL)) { errno = EINVAL; return OS_INVALID_HANDLE; }
    int fds[2];
    if (pipe(fds) < 0) return OS_INVALID_HANDLE;
    raii(os_file_handle) parent_read = fds[0];
    raii(os_file_handle) child_write = fds[1];
    for (int i = 0; i < 2; ++i) {
        int flags = fcntl(fds[i], F_GETFD);
        if (flags < 0 || fcntl(fds[i], F_SETFD, flags | FD_CLOEXEC) < 0)
            return OS_INVALID_HANDLE;
    }

    pid_t pid = fork();
    if (pid < 0) return OS_INVALID_HANDLE;
    if (pid == 0) {
        close((int)move(os_file_handle, parent_read));
```

```
        execv(path, argv);
        int child_errno = errno;
        (void)os_file_write_full(child_write, &child_errno, sizeof(child_errno));
        _exit(EXIT_FAILURE);
    }

    close((int)move(os_file_handle, child_write));
    int child_errno = 0;
    os_ssize_t bytes = os_file_read_full(parent_read, &child_errno, sizeof(child_errno));
    if (bytes != 0) {
        int saved_errno = bytes < 0 ? errno :
            bytes == (os_ssize_t)sizeof(child_errno) ? child_errno : EIO;
        int exit_code;
        if (bytes != (os_ssize_t)sizeof(child_errno)) os_process_terminate(pid);
        (void)os_process_wait(pid, &exit_code);
        errno = saved_errno;
        return OS_INVALID_HANDLE;
    }
    return pid;
}

static int wait_process(os_process_handle_t process, int *exit_code, int flags) {
    int status;
    pid_t result;
    do {
        result = waitpid((pid_t)process, &status, flags);
    } while (result < 0 && errno == EINTR);
    if (result <= 0) return (int)result;
    *exit_code = WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
    return 1;
}

int os_process_poll(os_process_handle_t process, int *exit_code) {
    return wait_process(process, exit_code, WNOHANG);
}

int os_process_wait(os_process_handle_t process, int *exit_code) {
    return wait_process(process, exit_code, 0);
}

void os_process_terminate(os_process_handle_t process) {
    if (process > 0) (void)kill((pid_t)process, SIGTERM);
}

os_process_handle_t os_process_id(void) { return getpid(); }
os_process_handle_t os_process_parent_id(void) { return getppid(); }
```

common/os/posix/shm.c

```
#include "common/os/shm.h"
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/mman.h>

static mode_t posix_mode(os_mode_t mode) {
    mode_t result = 0;
    if (mode & OS_S_IRUSR) result |= S_IRUSR;
    if (mode & OS_S_IWUSR) result |= S_IWUSR;
    if (mode & OS_S_IXUSR) result |= S_IXUSR;
    if (mode & OS_S_IRGRP) result |= S_IRGRP;
    if (mode & OS_S_IWGRP) result |= S_IWGRP;
    if (mode & OS_S_IXGRP) result |= S_IXGRP;
    if (mode & OS_S_IROTH) result |= S_IROTH;
    if (mode & OS_S_IWOTH) result |= S_IWOTH;
    if (mode & OS_S_IXOTH) result |= S_IXOTH;

    return result;
}

```

```c
os_shm_file_t os_shm_open(const char *name, os_o_flags_t flags, os_mode_t mode) {
    int oflag = 0;
    if (flags & OS_O_RDWR) {
        oflag |= O_RDWR;
    } else if (flags & OS_O_RDONLY) {
        oflag |= O_RDONLY;
    }

    if (flags & OS_O_CREAT) {
        oflag |= O_CREAT;
    }
    if (flags & OS_O_EXCL) oflag |= O_EXCL;
    if (flags & OS_O_TRUNC) {
        oflag |= O_TRUNC;
    }
    mode_t pmode = posix_mode(mode);
    os_shm_file_t fd = (os_shm_file_t)shm_open(name, oflag, pmode);
    return fd;
}

int os_shm_unlink(const char *name) {
    return shm_unlink(name);
}
```

common/os/process.h

```c
#ifndef OS_PROCESS_H
#define OS_PROCESS_H

#include "common/os/common.h"

os_process_handle_t os_process_spawn(const char *path, char *const argv[]);
int os_process_poll(os_process_handle_t process, int *exit_code);
int os_process_wait(os_process_handle_t process, int *exit_code);
void os_process_terminate(os_process_handle_t process);
os_process_handle_t os_process_id(void);
os_process_handle_t os_process_parent_id(void);

#endif
```

common/os/shm.h

```c
#ifndef SHM_H
#define SHM_H

#include <stdbool.h>
#include <stddef.h>

#include "common/os/common.h"

#define SHM_NAME_LEN 127

typedef os_file_handle_t os_shm_file_t;

typedef enum {
    OS_O_RDONLY = 1,
    OS_O_RDWR = 1 << 2,
    OS_O_CREAT = 1 << 3,
    OS_O_TRUNC = 1 << 4,
    OS_O_EXCL = 1 << 5,
} os_o_flags_t;

typedef enum {
    OS_S_IRUSR = 00400,
    OS_S_IWUSR = 00200,
    OS_S_IXUSR = 00100,
```

```
    OS_S_IRGRP = 00040,
    OS_S_IWGRP = 00020,
    OS_S_IXGRP = 00010,
    OS_S_IROTH = 00004,
    OS_S_IWOTH = 00002,
    OS_S_IXOTH = 00001,
} os_mode_t;

os_shm_file_t os_shm_open(const char *name, os_o_flags_t flags, os_mode_t mode);
int os_shm_unlink(const char *name);

#endif //SHM_H
```

common/unlikely.h

```
#ifndef UNLIKELY_H
#define UNLIKELY_H
#if defined(__GNUC__) || defined(__clang__)
#define unlikely(x) __builtin_expect(!!(x), 0)
#define likely(x)   __builtin_expect(!!(x), 1)
#else

#define unlikely(x) (x)
#define likely(x)   (x)
#endif

#endif //UNLIKELY_H
```